

Documentación Técnica — Performance & Load Testing con k6

Documentación de referencia del diseño, las decisiones técnicas y el funcionamiento del proyecto, incluyendo la interpretación de métricas y la integración en el pipeline.

Contenido

1. Alcance y responsabilidad
2. Los tipos de prueba de rendimiento
3. Diseño de carga realista
4. Thresholds como quality gate
5. Interpretación de métricas
6. El servicio bajo prueba
7. Integración continua
8. Cómo extender el proyecto
9. Glosario

1. Alcance y responsabilidad

El testing funcional verifica que el sistema **hace** lo correcto; el testing de rendimiento verifica que lo hace **a escala y con la latencia esperada**. Un sistema puede funcionar perfectamente con un usuario y colapsar con mil. Estas pruebas responden preguntas que el testing funcional no aborda: cuánta carga soporta, con qué tiempos de respuesta, y cómo se comporta al superarla.

Responsabilidad: las pruebas se ejecutan contra un servicio bajo prueba **local y propio**. Ejecutar pruebas de carga sobre un servicio de terceros sin autorización explícita es, técnicamente, un ataque de denegación de servicio. Probar contra un SUT propio es la práctica correcta, no tiene impacto externo y da control total sobre el entorno medido.

2. Los tipos de prueba de rendimiento

Cada tipo responde una pregunta de riesgo distinta sobre el mismo sistema:

Tipo	Carga	Qué responde
Smoke	1 usuario, breve	¿El script y los endpoints funcionan? ¿La latencia base es razonable?
Load	La esperada en producción	¿Se cumplen los SLOs en condiciones normales?
Stress	Por encima de lo esperado	¿Cuál es el punto de quiebre y cómo se degrada?
Spike	Pico súbito	¿Resiste una avalancha repentina y se recupera?
Soak	Media, sostenida en el tiempo	¿Se degrada con las horas (fugas de memoria, recursos)?

En k6, la carga se modela con **stages**: cada stage lleva la cantidad de usuarios virtuales (VUs) a un **target** durante una **duration**. Encadenando stages se construye el perfil de cada tipo (ramp-up, sostenido, ramp-down, o un pico).

3. Diseño de carga realista

Una prueba de carga solo sirve si el escenario se parece al tráfico real. El modelo de usuario (`lib/flow.js`) aplica tres principios:

1. **Mezcla de operaciones realista.** No todos los usuarios hacen lo mismo. Acá, todos navegan el catálogo y solo una fracción (20 %) concreta una compra, reproduciendo la proporción típica de un e-commerce.
2. **Think times.** Un usuario real hace pausas entre acciones; no dispara requests sin parar. Se incluye un `sleep` de 0,5–1,5 s. Omitirlo mediría un escenario irreal y saturaría artificialmente el sistema.
3. **Ramp-up gradual.** La carga sube de a poco (0 → 20 usuarios en 10 s), no toda de golpe. Esto refleja cómo crece el tráfico real y permite observar en qué punto empieza la degradación.

El error clásico que esto evita: lanzar 1.000 iteraciones idénticas sin pausa mediría la performance del caché, no la del sistema real.

4. Thresholds como quality gate

Es la decisión central del proyecto. Los SLOs (Service Level Objectives) se expresan como **thresholds**, y k6 termina con **código de salida distinto de cero** si no se cumplen:

```
thresholds: {
  http_req_failed: ['rate<0.01'],
  http_req_duration: ['p(95)<500', 'p(99)<800'],
}
```

Gracias a esto, una **regresión de rendimiento bloquea el pipeline** exactamente igual que un test funcional roto: si un cambio de código hace que el p95 pase de 40 ms a 600 ms, el build falla y el problema se detecta antes de producción.

El orquestador (`scripts/run.mjs`) propaga ese código de salida: levanta el servicio, corre k6, y si k6 falla por un threshold incumplido, el proceso también falla. Este comportamiento se validó en ambos sentidos: cumple cuando el sistema respeta el SLO, y falla cuando se le impone un umbral imposible.

5. Interpretación de métricas

Saber leer los resultados es tan importante como generarlos.

- **Latencia en percentiles (p95, p99), no promedio.** El promedio esconde la cola: un promedio de 200 ms puede ocultar que el 5 % de los usuarios espera 8 segundos. El p95 dice "el 95 % de los usuarios experimentó esto o menos". Por eso los SLOs se fijan sobre percentiles.
- **Throughput (`http_reqs/s`).** Cuántas requests por segundo procesa el sistema. Mide capacidad.
- **Tasa de error (`http_req_failed`).** Proporción de requests fallidas. Bajo carga normal debe ser cercana a cero.
- **Saturación de recursos.** En un sistema real, se cruza la degradación de latencia con CPU, memoria, conexiones de base de datos y profundidad de colas para **ubicar el cuello de botella**. Si la latencia se dispara cuando el pool de conexiones llega al 100 %, el cuello no es la CPU sino las conexiones — y optimizar el lugar equivocado no sirve.

El objetivo no es solo saber "cuánto aguanta", sino "qué se rompe primero y por qué".

6. El servicio bajo prueba

`service/server.mjs` es un API mínimo en Node sin dependencias, con tres endpoints (`/health` , `/products` , `/orders`) y una latencia artificial pequeña y variable (10–40 ms) para que las métricas de percentiles sean representativas.

Es deliberadamente liviano y autocontenido: el foco del proyecto es la **metodología de performance testing**, no el sistema medido. Como es un servicio simple, tolera con holgura incluso el stress a 200 usuarios; en un sistema real, el stress test revelaría el punto de quiebre. Cambiar el objetivo a un servicio real es tan simple como apuntar `BASE_URL` a su URL (con la debida autorización).

7. Integración continua

El pipeline ([.github/workflows/ci.yml](#)) instala k6, y en cada push/PR ejecuta el **smoke** y el **load** test. Los thresholds actúan como gate: si el rendimiento se degrada por debajo del SLO, el job falla.

Los escenarios de **stress, spike y soak** no se corren en cada PR (son más largos y su objetivo es el análisis, no el gate): se ejecutan bajo demanda o en corridas programadas, siguiendo la misma lógica de dos velocidades del pipeline de CI/CD de la serie.

8. Cómo extender el proyecto

- **Exportar a un dashboard:** enviar las métricas de k6 a Grafana + Prometheus/InfluxDB para visualizar tendencias en el tiempo.
- **Más escenarios de negocio:** modelar flujos con autenticación, carritos de varios ítems, o distintos perfiles de usuario con `scenarios` de k6.
- **Umbrales por endpoint:** definir SLOs distintos para operaciones distintas (una lectura debe ser más rápida que una escritura).
- **Correlación con recursos:** instrumentar el SUT para exponer métricas de CPU/memoria y cruzarlas con la latencia.
- **Gate programado:** ejecutar el load test en una corrida nocturna, además del smoke en cada PR.

9. Glosario

- **Performance testing:** verificación del comportamiento del sistema bajo carga (capacidad, latencia, estabilidad).
- **VU (Virtual User):** usuario virtual; una unidad de concurrencia que k6 simula.
- **Stage:** tramo de carga con un objetivo de VUs y una duración; se encadenan para formar el perfil de la prueba.
- **Threshold:** umbral (SLO) que, si no se cumple, hace fallar la prueba. Es el quality gate.
- **SLO (Service Level Objective):** objetivo medible de nivel de servicio (ej: $p95 < 500 \text{ ms}$).
- **p95 / p99:** percentiles de latencia; el 95 % / 99 % de las requests respondió en ese tiempo o menos.
- **Throughput:** requests procesadas por unidad de tiempo.
- **Think time:** pausa entre acciones que reproduce el comportamiento de un usuario real.
- **Ramp-up / ramp-down:** subida / bajada gradual de la carga.
- **SUT (System Under Test):** el sistema que se está probando.

- **Saturación:** nivel de uso de un recurso (CPU, memoria, conexiones); su cruce con la latencia ubica el cuello de botella.